

Aula de hoje: gerador, blocos e painel do armazém

24/09/2026 · Instrutor Pedro Leonardo

O que vamos fazer hoje

Hoje o dinheiro do tycoon deixa de aparecer sozinho: agora ele vem de blocos que caem de uma máquina. É o começo do sistema de dropper.

- **Moldes de bloco:** um bloco modelo guardado fora do mapa, para ser copiado.
- **Script do Bloco:** cada bloco sabe quanto vale e some sozinho depois de um tempo.
- **Tipos de bloco:** blocos diferentes valem valores diferentes.
- **Gerador:** a máquina que cria um bloco a cada poucos segundos.
- **Painel do armazém:** uma placa que mostra quanto dinheiro está guardado.

No fim da aula você vai ver blocos caindo do gerador e o valor do armazém aparecendo em uma placa no mapa.

Uma regra para todos os scripts: as configurações (valor, tempo, velocidade) ficam em variáveis no topo do script. Quer mudar o jogo? Mude só essas linhas.

Etapa 1: moldes de bloco

Por quê: o gerador precisa de um modelo para copiar, como um carimbo. Guardamos o modelo no `ServerStorage` porque o que fica lá não aparece no mapa e não é afetado pela física.

Como fazer:

- No Explorer, clique com o botão direito em `ServerStorage` e insira uma **Folder**. Dê o nome de `Blocos`.
- No WorksPace crie um bloco, customize e depois o passe para a pasta que acabou de criar e dê o nome de `BlocoComum`.
- Nas propriedades da Part:
 - Size: 1, 1, 1 (um cubo pequeno)
 - Anchored: **desmarcado** (o bloco precisa cair)
 - CanCollide: **marcado** (o bloco precisa bater nas coisas)

O nome `Blocos` precisa ser escrito exatamente assim, com B maiúsculo, porque é com esse nome que o gerador procura a pasta.

Etapa 2: script do Bloco

Por quê: cada bloco precisa guardar duas informações: quanto vale e quanto tempo pode existir. O tempo de vida é importante porque um bloco que cai para fora da esteira ficaria no mapa para sempre, e com centenas de blocos o jogo começa a travar.

Como fazer: dentro de `BlocoComum`, insira um **Script** chamado `Bloco` e cole o código:

```
-- =====
-- SCRIPT: BLOCO
-- O "material" que cai do gerador e vale dinheiro
-- Local: ServerStorage.Blocos.BlocoComum.Bloco
-- =====

local bloco = script.Parent

-- =====
-- CONFIGURAÇÕES DO BLOCO
-- =====
local valor = 5          -- quanto dinheiro esse bloco vale
local tempoDeVida = 20   -- segundos até o bloco sumir sozinho

-- guarda o valor no próprio bloco para outros scripts lerem
bloco:SetAttribute("Valor", valor)

-- o Debris destrói o bloco depois de X segundos
local Debris = game:GetService("Debris")
Debris:AddItem(bloco, tempoDeVida)
```

Entendendo o código:

- `script.Parent` é o objeto onde o script está, ou seja, o próprio bloco.
- `SetAttribute` escreve o valor no bloco, como uma etiqueta de preço. Um script não enxerga as variáveis de outro, mas consegue ler essa etiqueta. Na próxima aula, o `DestinoFinal` vai lê-la.
- `Debris` é o "lixeiro" do Roblox: você diz o que jogar fora e daqui a quantos segundos.

Este script não roda enquanto está no `ServerStorage`. Ele só começa a rodar quando o gerador coloca uma cópia do bloco no mapa.

Etapa 3: tipos de bloco

Por quê: em um tycoon, as máquinas mais caras produzem blocos que valem mais. Ter tipos diferentes dá ao jogador um motivo para continuar juntando dinheiro.

Como fazer:

- Selecione `BlocoComum` e aperte **Ctrl+D** para duplicar.
- Renomeie a cópia para `BlocoRaro`.
- Mude a cor ou o material, para o jogador perceber a diferença.
- Abra o script `Bloco` da cópia e troque só a linha do valor, por exemplo `local valor = 20`.

Exemplo de tabela para a turma montar:

Bloco	Cor	Valor
BlocoComum	Cinza	5
BlocoRaro	Azul	20
BlocoEpico	Dourado	50

Etapa 4: o Gerador

Por quê: o gerador é a máquina do tycoon. Ele copia o molde a cada poucos segundos e solta o bloco logo abaixo dele. Ele só funciona depois de comprado, por isso fica dentro da pasta de um botão de compra.

Como fazer:

- Dentro da pasta do botão que vai vendê-lo, insira uma **Part** chamada Gerador.
- Marque **Anchored** (a máquina não pode cair) e deixe **Transparency** em 1 e o **cancollision** desativado, igual aos outros itens que estão à venda.
- Dentro de Gerador, insira um **Script** chamado Gerador e cole o código:

```
-- =====
-- SCRIPT: GERADOR (DROPPER)
-- Cria um bloco a cada X segundos embaixo do gerador
-- Local: dentro da peça Gerador
-- =====

local ServerStorage = game:GetService("ServerStorage")

-- a própria peça do gerador
local gerador = script.Parent

-- =====
-- CONFIGURAÇÕES DO GERADOR
-- =====
local tipoDeBloco = "BlocoComum" -- nome do molde dentro de ServerStorage.Blocos
local intervaloCriacao = 2 -- segundos entre cada bloco

-- pega o molde do bloco escolhido
local molde = ServerStorage.Blocos:WaitForChild(tipoDeBloco)

-- =====
-- FUNÇÃO: cria um bloco novo embaixo do gerador
-- =====
local function criarBloco()
    local bloco = molde:Clone() -- faz uma cópia do molde

    -- posiciona logo abaixo do gerador
    local distancia = gerador.Size.Y / 2 + bloco.Size.Y / 2 + 0.5
    bloco.CFrame = gerador.CFrame * CFrame.new(0, -distancia, 0)
```

```

    bloco.Anchored = false -- deixa a física agir (o bloco cai)
    bloco.Parent = workspace -- coloca no mundo, e o script Bloco começa a rodar

    -- o servidor controla a física do bloco (evita que ele "trema")
    bloco:SetNetworkOwner(nil)
end

-- =====
-- LOOP INFINITO: cria blocos para sempre
-- =====
while true do
    task.wait(intervaloCriacao)

    -- só gera se o gerador já foi comprado
    -- (o botão deixa a peça visível ao comprar)
    if gerador.Transparency < 1 then
        criarBloco()
    end
end
end

```

Entendendo o código:

- `tipoDeBloco` escolhe qual molde copiar. Para um gerador de blocos raros, troque para `"BlocoRaro"`.
- `Clone()` faz uma cópia do molde, como passar o carimbo no papel.
- A conta da `distancia` coloca o bloco logo abaixo do gerador, sem que os dois fiquem um dentro do outro.
- `bloco.Parent = workspace` é o momento em que o bloco aparece no mapa.
- `while true do` repete para sempre. O `task.wait` é a pausa entre um bloco e outro.
- `if gerador.Transparency < 1` confere se o gerador já foi comprado: se ele ainda está invisível, não solta nada.

Os blocos vão para o `workspace`, e não para a pasta do botão. Se ficassem na pasta, o script do botão prenderia (ancoraria) todos eles no momento da compra.

Etapa 5: painel do armazém

Por quê: até agora o dinheiro só aparecia no Output, que o jogador nunca vê. Com o painel, qualquer um olha para o mapa e sabe quanto está esperando para ser coletado.

Como fazer:

- No `Workspace`, insira uma **Part** chamada `PainelArmazem`. Marque **Anchored** e estique no formato de placa (por exemplo 6, 3, 1).
- Dentro dela, insira um **SurfaceGui**. Em `Face`, escolha o lado onde o texto vai aparecer (`Front`) e mude `SizingMode` para **PixelsPerStud**.

- Dentro do SurfaceGui, insira um **TextLabel**. Coloque Size em {1,0},{1,0} (ocupa a placa inteira), marque TextScaled e escolha as cores.
- Dentro de PainelArmazem, insira um **Script** chamado PainelArmazem e cole o código:

```
-- =====
-- SCRIPT: PAINEL DO ARMAZÉM
-- Mostra num bloco quanto dinheiro está guardado no armazém
-- Local: Workspace.PainelArmazem
-- =====

-- pega o módulo Economia (o mesmo que o Gerador e o Coletor usam)
local ServerScriptService = game:GetService("ServerScriptService")
local Economia = require(ServerScriptService.Economia)

-- o bloco onde o script está e o texto que fica dentro dele
local painel = script.Parent
local texto = painel:WaitForChild("SurfaceGui"):WaitForChild("TextLabel")

-- =====
-- CONFIGURAÇÕES DO PAINEL
-- =====
local intervaloAtualizacao = 0.5 -- de quantos em quantos segundos o texto atualiza
local prefixo = "Armazém: $" -- o que aparece antes do número

-- =====
-- LOOP INFINITO: atualiza o texto para sempre
-- =====
while true do
    -- junta o prefixo com o valor atual do armazém
    texto.Text = prefixo .. Economia.armazem
    task.wait(intervaloAtualizacao)
end
```

Entendendo o código:

- require pega o módulo Economia, onde o valor do armazém está guardado.
- WaitForChild espera o objeto existir antes de usá-lo, para o script não quebrar se ele carregar primeiro.
- .. junta dois textos: "Armazém: \$" com o número vira Armazém: \$40.
- O loop confere o armazém duas vezes por segundo e reescreve o texto.

Para o texto flutuar acima do bloco e ficar sempre virado para o jogador, use um **BillboardGui** no lugar do SurfaceGui: Size {0,200},{0,50} e StudsOffset 0, 3, 0. No script, troque "SurfaceGui" por "BillboardGui".

Testar e problemas comuns

Dê **Play**, compre o gerador e confira:

- ☐ Os blocos caem do gerador a cada 2 segundos

- ☐ Os blocos somem sozinhos depois de 20 segundos
- ☐ O painel mostra o valor do armazém subindo

Problema	Causa provável	Solução
Nenhum bloco aparece	O gerador ainda está invisível ou o nome do molde está errado	Compre o gerador e confira se tipoDeBloco é igual ao nome da Part em Blocos
Erro "Blocos is not a valid member"	A pasta tem outro nome	Renomeie a pasta para Blocos, exatamente assim
Uma peça ancorada continua caindo	O Anchored foi marcado durante o Play, e o que muda no teste é desfeito no Stop	Pare o jogo e marque Anchored de novo
O texto do painel não aparece	O SurfaceGui está em outra face do bloco	Troque a propriedade Face (Front, Back, Top...)
Os blocos ficam presos no gerador	O bloco nasce dentro do gerador	Aumente o 0.5 na conta da distancia

Próximas aulas

- **Esteira:** empurra os blocos e o jogador, com velocidade configurável.
- **DestinoFinal:** lê a etiqueta Valor do bloco, faz o bloco sumir e gera o dinheiro.
- **Botão para pegar a base:** define quem é o dono de cada tycoon.

Tarefa em aula

- **Faça toda a implementação do arquivo**
- **Faça o máximo possível da base com os recursos que tem**
- **Deixe o texto do coletor bonito**